

REFORÇO · REDES NEURAIS ARTIFICIAIS

Do problema ao neurônio

Aula expositiva · acompanha o notebook Reforco_RedesNeurais

Roteiro da aula

- 1 Definição do problema
- 2 Primeira abordagem: Regressão Linear
- 3 Segunda abordagem: Regressão Polinomial
- 4 Uma abordagem alternativa: rampas
- 5 Soma de rampas
- 6 Ajuste automático dos valores
- 7 Terminologia: neurônio, camada e pesos
- 8 Implementação com o Scikit-Learn
- 9 Síntese



Como esta aula está organizada

- Partimos de um problema concreto, antes de qualquer termo técnico.
- Testamos primeiro o que já conhecemos (Regressão Linear e Regressão Polinomial) e observamos onde falham.
- Construimos uma alternativa e só então lhe damos nome: neurônio, camada e pesos.

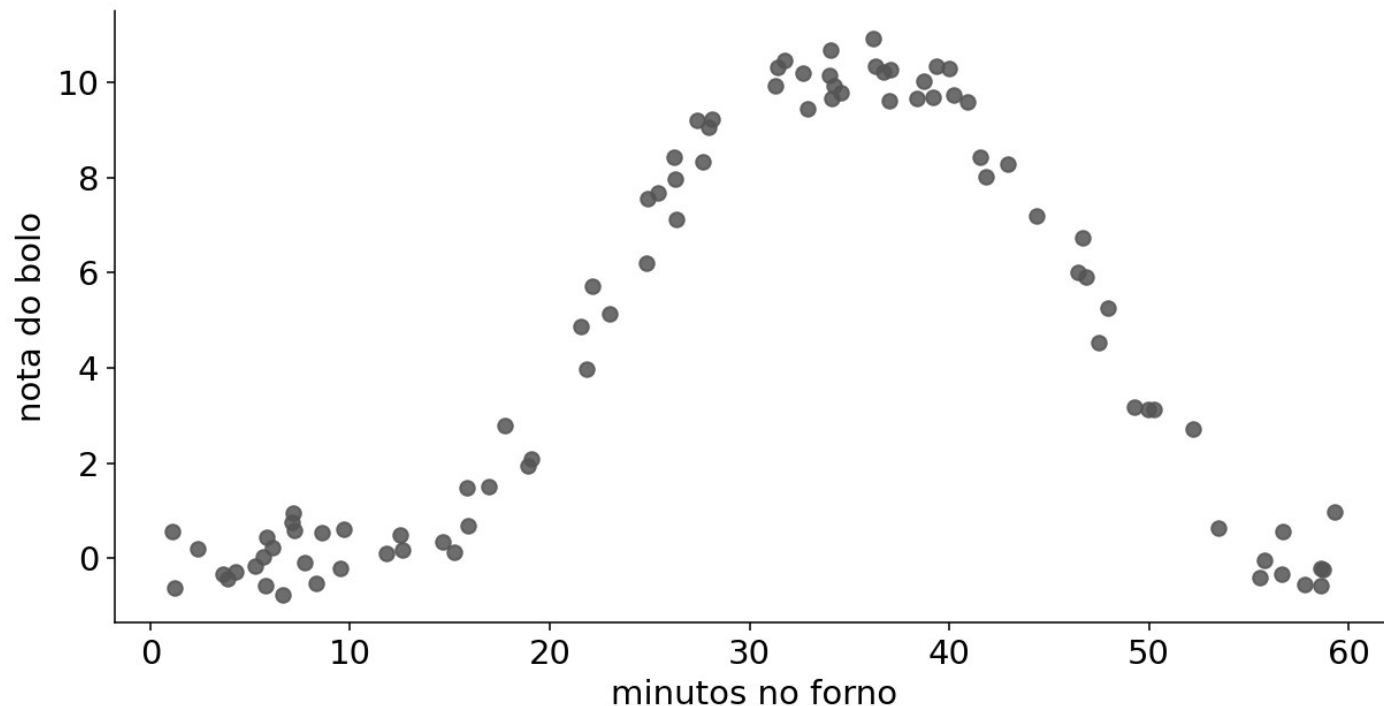
Cada passo desta aula corresponde a um passo do notebook.

O experimento: tempo no forno e nota do bolo

Tempo no forno	O que acontece	Nota
Curto	A massa fica crua	Baixa
Adequado	O bolo fica no ponto	Alta
Excessivo	O bolo queima	Baixa

Objetivo: prever a nota a partir do tempo no forno. Como o valor previsto é numérico, trata-se de um problema de regressão.

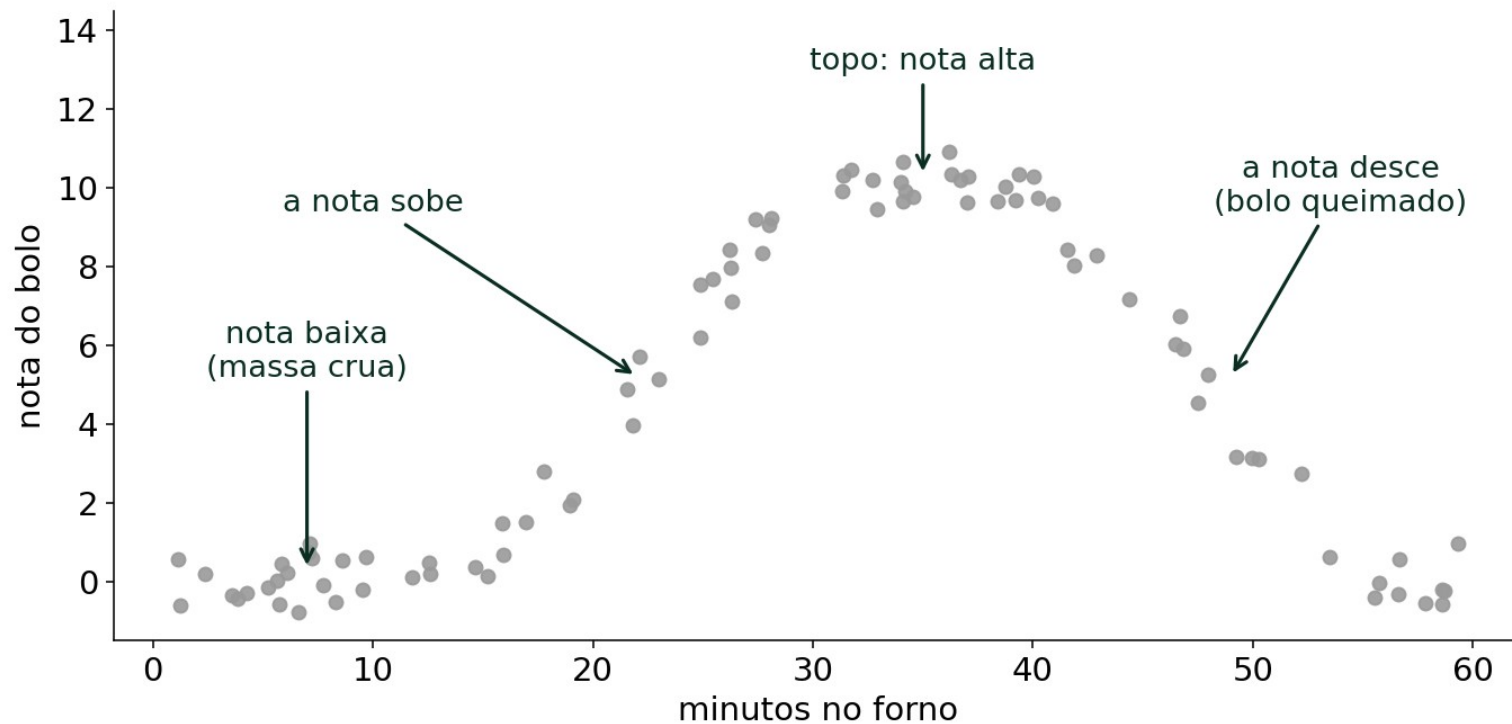
Os dados do experimento



- 90 bolos assados por tempos diferentes
- Cada ponto é um bolo
- Eixo horizontal: minutos no forno
- Eixo vertical: nota recebida

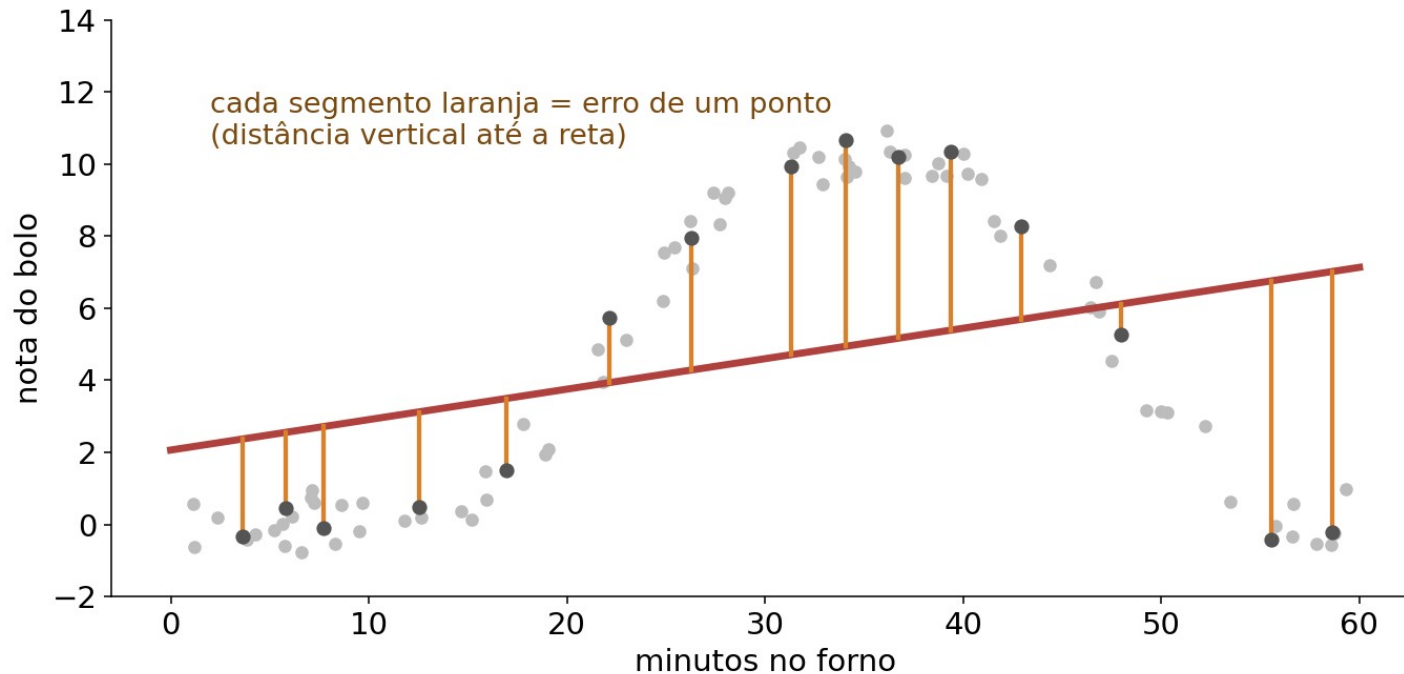


O formato dos dados



- Nota baixa até cerca de 15 minutos
- Sobe e permanece alta entre cerca de 30 e 40 minutos
- Volta a cair quando o tempo é excessivo

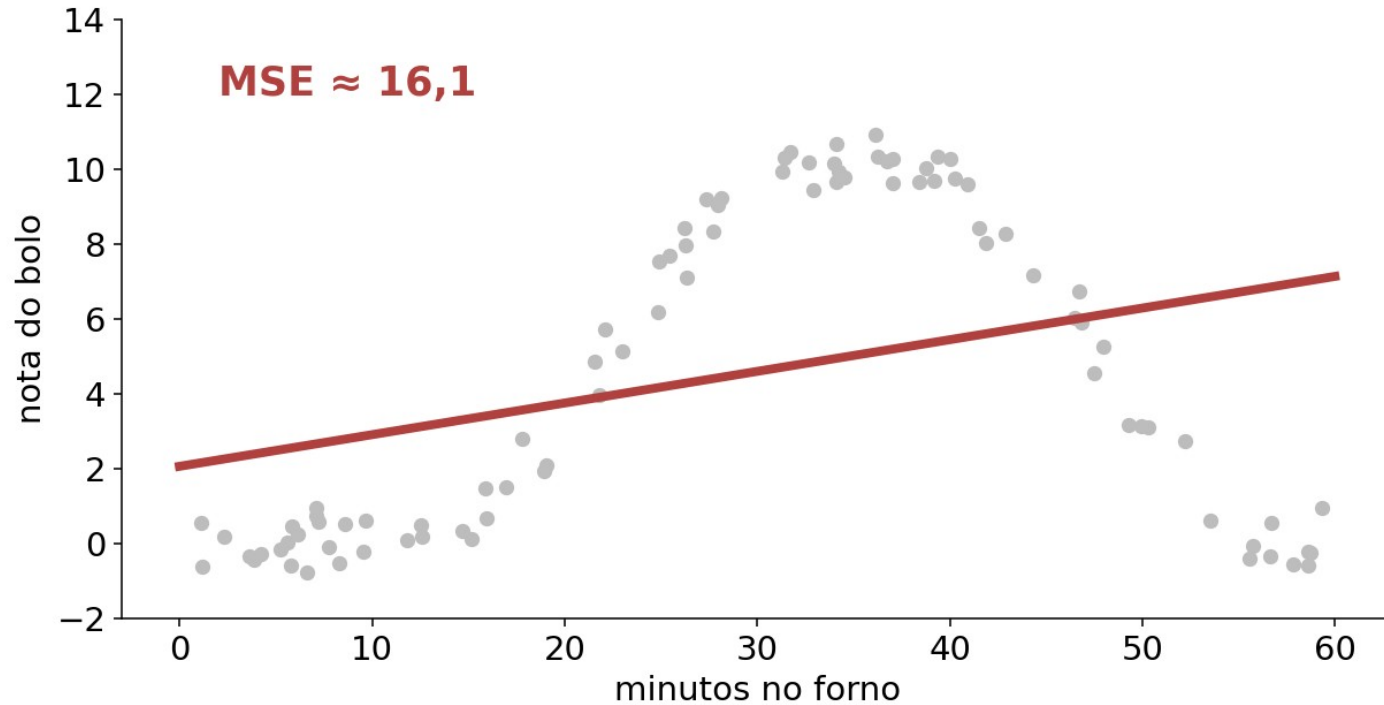
Como medir o erro: o MSE



1. Para cada ponto, mede-se a distância vertical até a curva: o erro.
2. Cada erro é elevado ao quadrado.
3. O MSE é a média desses quadrados.

Quanto menor o MSE, melhor o ajuste.

Primeira abordagem: a Regressão Linear



- A reta que minimiza o erro
- **MSE ≈ 16,1**
- A reta só sobe ou só desce

Prevê que mais tempo de forno é sempre melhor (ou sempre pior).

Segunda abordagem: curvar a reta

A Regressão Polinomial inclui potências do tempo (t^2 , t^3 , ...) para permitir curvas. O grau é a maior potência utilizada.

grau 1: $\text{nota} = a \cdot t + b$

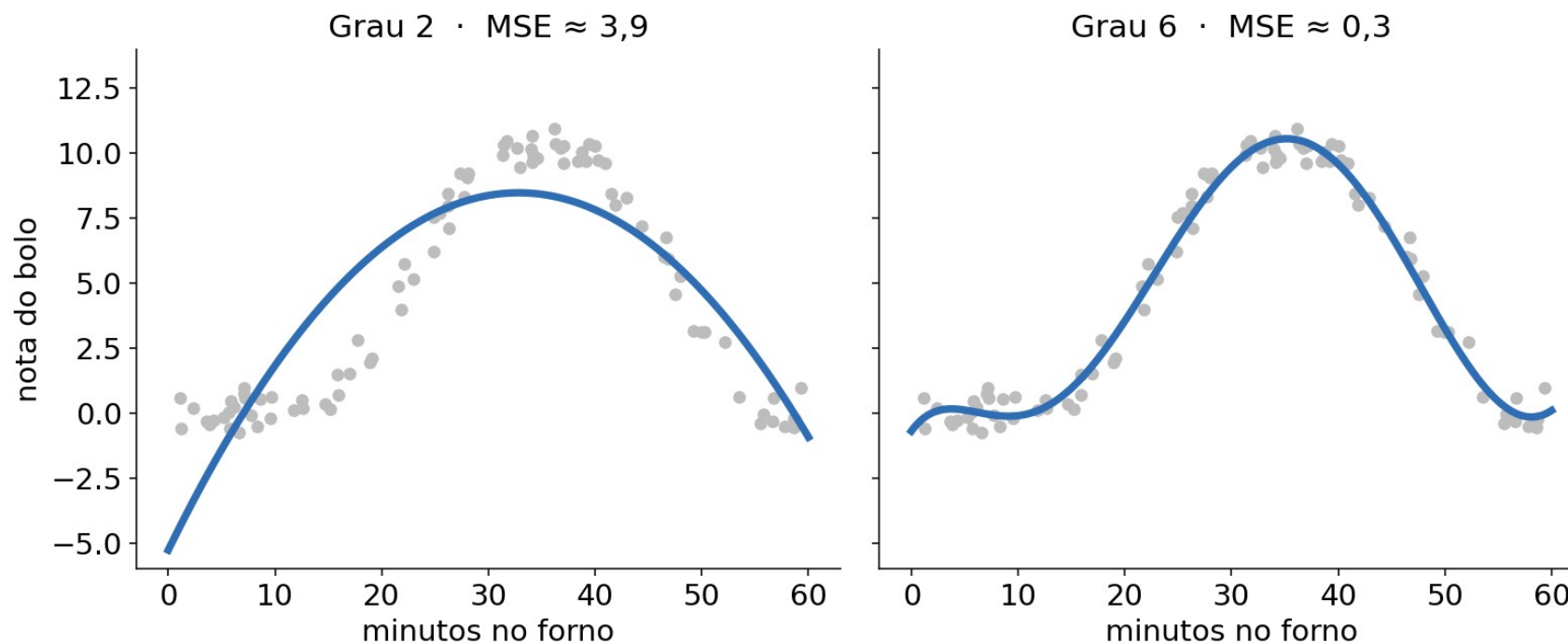
grau 2: $\text{nota} = a \cdot t^2 + b \cdot t + c$

grau 6: $\text{nota} = a_1 \cdot t + a_2 \cdot t^2 + \dots + a_6 \cdot t^6 + c$

Serão avaliados o grau 2 e o grau 6.

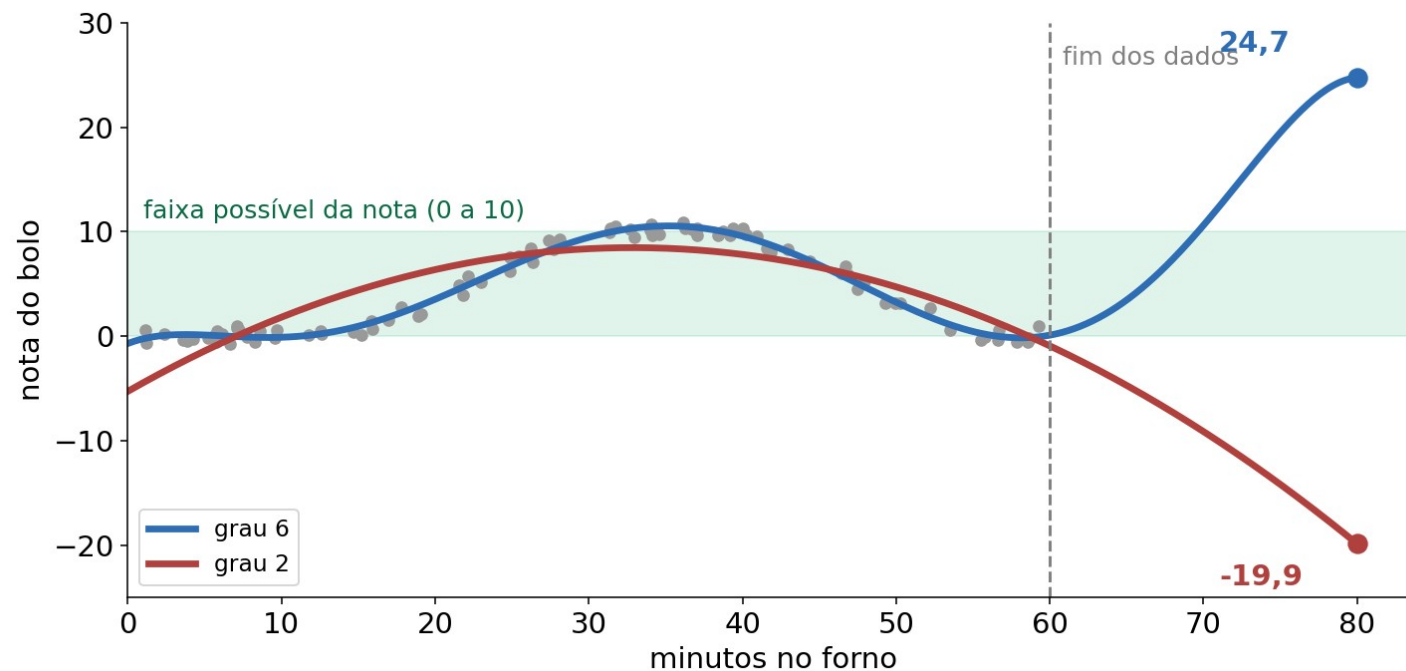


Grau 2 e grau 6



O grau 2 não acompanha o topo achatado; o grau 6 acompanha bem. Dentro da faixa observada, o polinômio resolve o problema.

Limitação 1: previsões fora da faixa dos dados



- Os dados cobrem de 0 a 60 minutos
- Para 80 minutos: grau 6 prevê $\approx 24,7$ e grau 2 prevê $\approx -19,9$
- Notas impossíveis em uma escala de 0 a 10

Fora da faixa observada, o polinômio diverge.

Limitação 2: o número de colunas cresce muito rápido

Regressão Polinomial de grau 3, com várias variáveis de entrada:

Variáveis de entrada	Colunas geradas
2	9
5	55
10	285
20	1770

Com muitas variáveis de entrada, a abordagem se torna progressivamente menos prática.

Conclusão do passo

O que o polinômio resolve

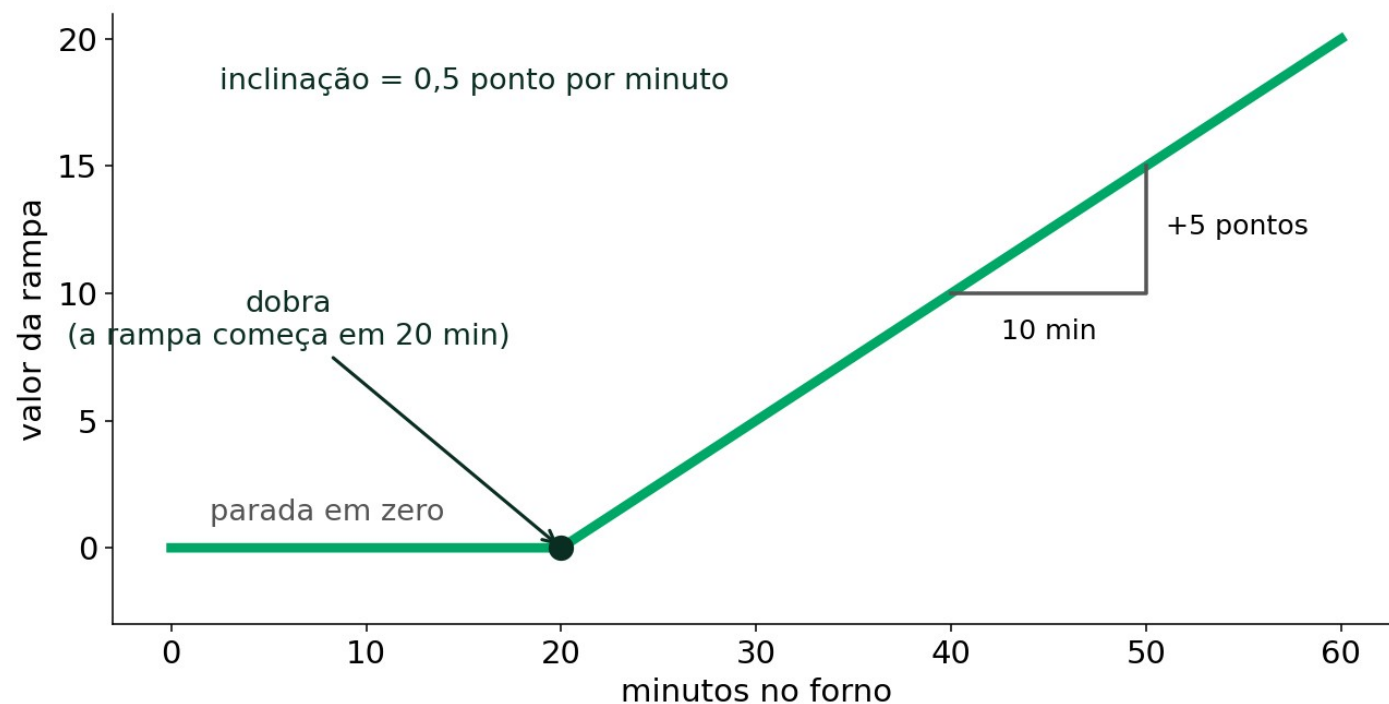
- Uma única entrada
- Dentro da faixa observada
- Grau 6: $MSE \approx 0,3$

Onde ele falha

- Diverge fora da faixa dos dados
- Escala mal com o número de entradas: 285 colunas com 10 entradas

É necessária outra ideia: emendar pedaços retos.

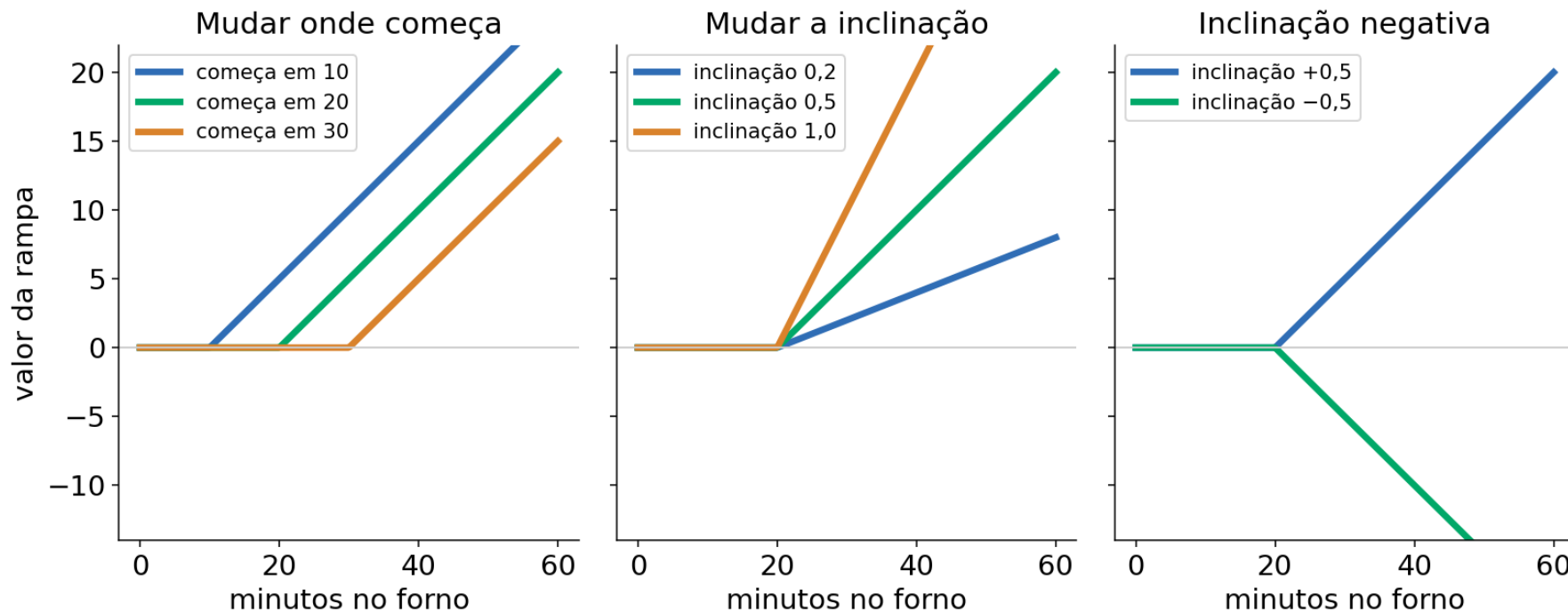
A rampa: a peça básica



- Valor zero até determinado minuto; a partir dele, cresce (ou decresce) em linha reta
- **Dois números:** onde começa e inclinação
- **Dobra:** o ponto em que a rampa deixa de ser parada



O efeito dos dois números



Maior onde_começa: a dobra se desloca para a direita

Maior inclinação: a rampa fica mais íngreme

Inclinação negativa: a rampa desce



No notebook: a função rampa

```
def rampa(t, onde_comeca, inclinacao):  
    return inclinacao * np.maximum(0, t - onde_comeca)
```

```
mostrar_rampa(20, 0.5) # altere estes 2 numeros
```

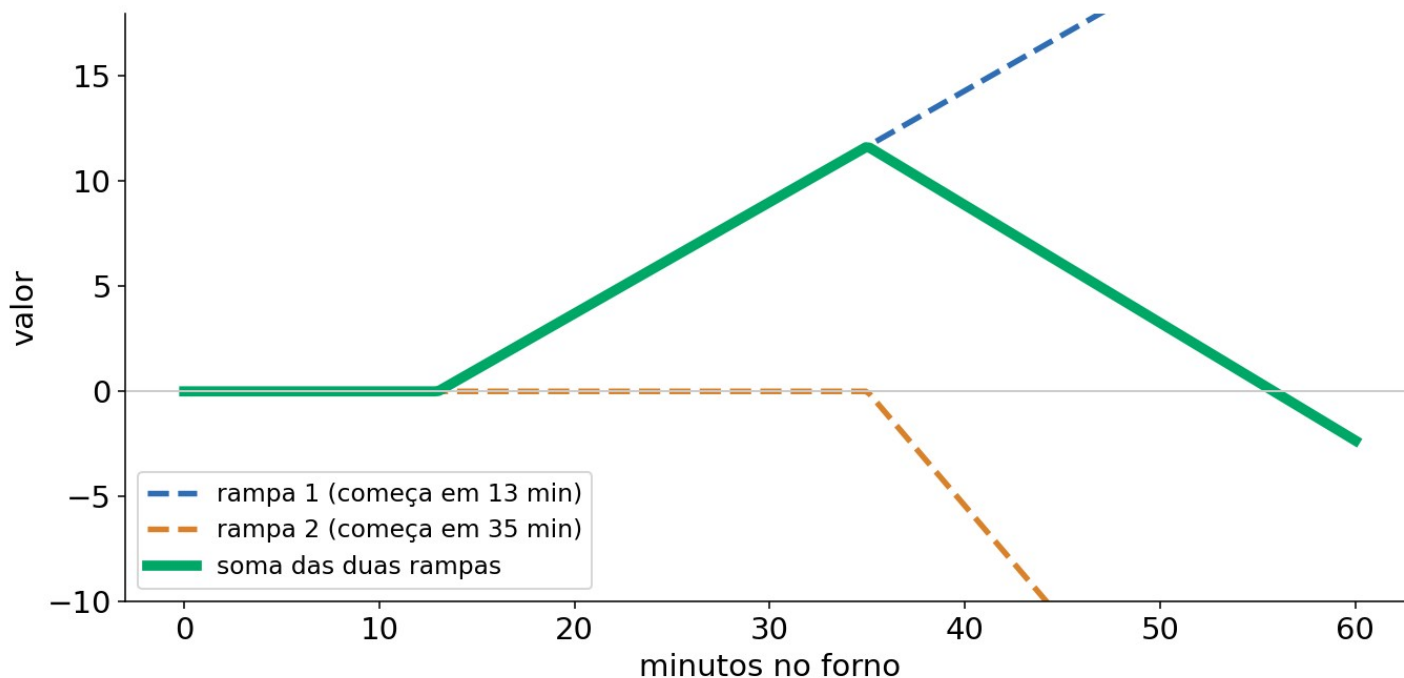
A função recebe o tempo e devolve o valor da rampa.

Atividade

- Altere os dois valores e execute novamente
- Maior onde_comeca: a dobra se desloca para a direita
- Maior inclinação: rampa mais íngreme
- Inclinação negativa: a rampa desce



Somando rampas



- Cada rampa adicionada cria uma nova dobra
- A soma das rampas forma a curva
- **Uma curva com n dobras tem $n + 1$ trechos retos**

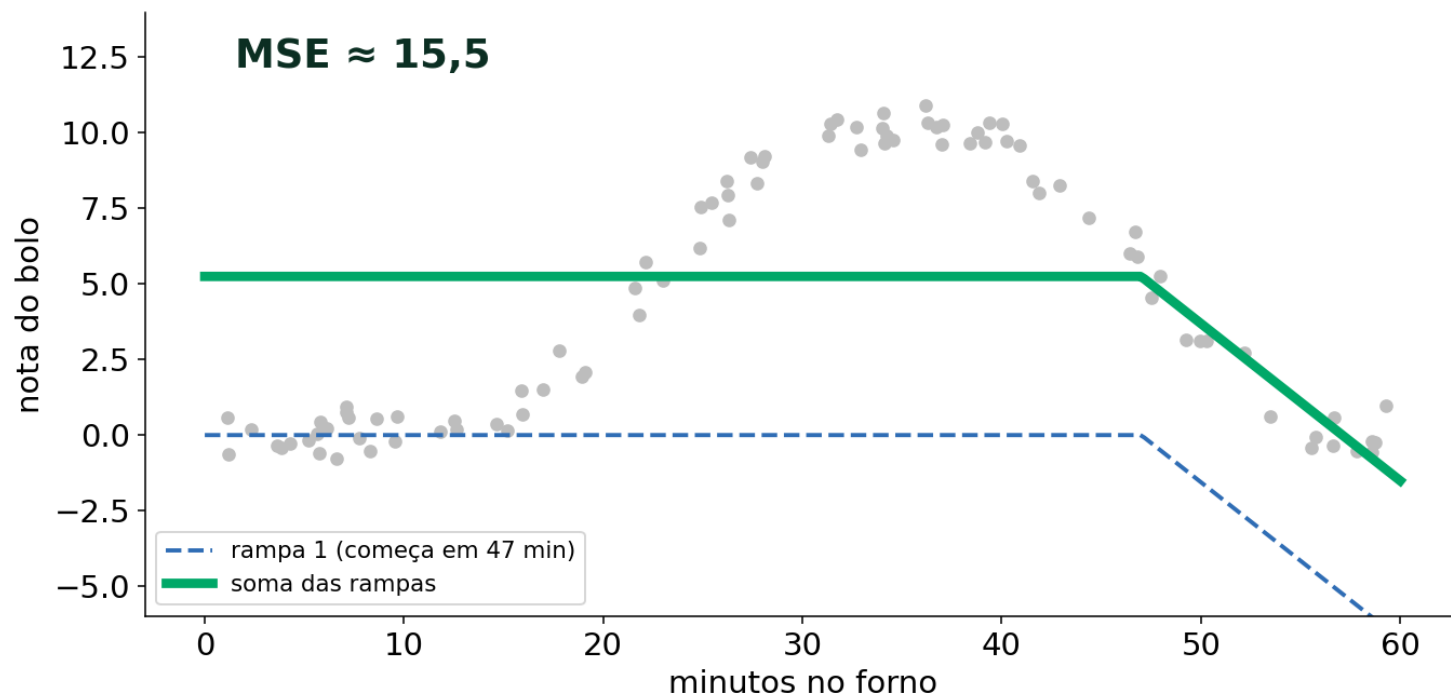
Neste exemplo: 2 rampas, 2 dobras, 3 trechos.

Atividade: ajuste manual de duas rampas

- 1 Estime em que minuto a nota começa a subir (cerca de 13 a 15 min): é o onde_comeca da primeira rampa.
- 2 Estime a inclinação da subida: cerca de 10 pontos em 20 minutos, ou seja, aproximadamente 0,5 ponto por minuto.
- 3 A segunda rampa deve começar no pico da montanha (cerca de 35 minutos).
- 4 A segunda rampa é somada à primeira: a partir de onde ela começa, as inclinações se somam. Para descer com inclinação de cerca de $-0,55$, a segunda deve ser aproximadamente $-0,55 - 0,5 = -1,05$.

Executar no notebook (Passo 5). Objetivo: MSE inferior a 3.

Uma dobra: 2 trechos retos



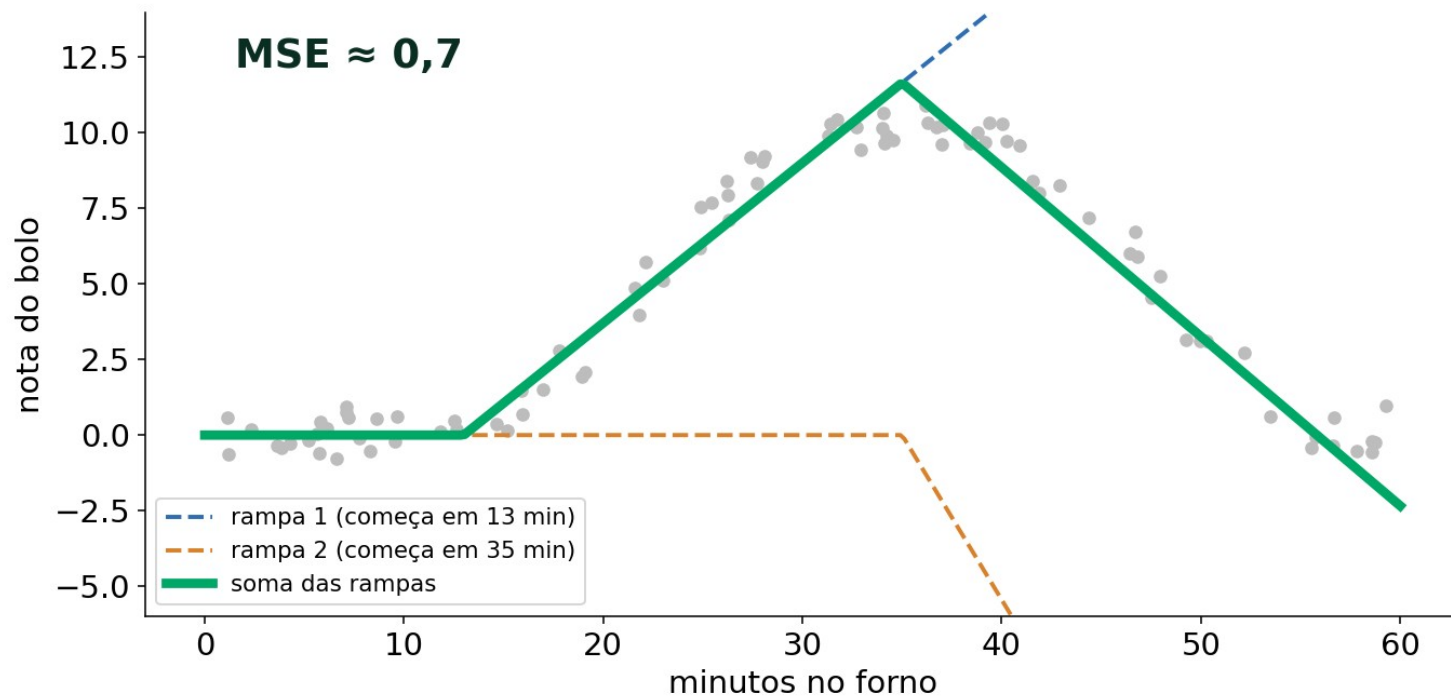
erro (MSE)

≈ 15,5

- Praticamente igual ao da reta (≈ 16,1)
- A curva muda de direção uma única vez
- A montanha exige subir e depois descer



Duas dobras: 3 trechos retos



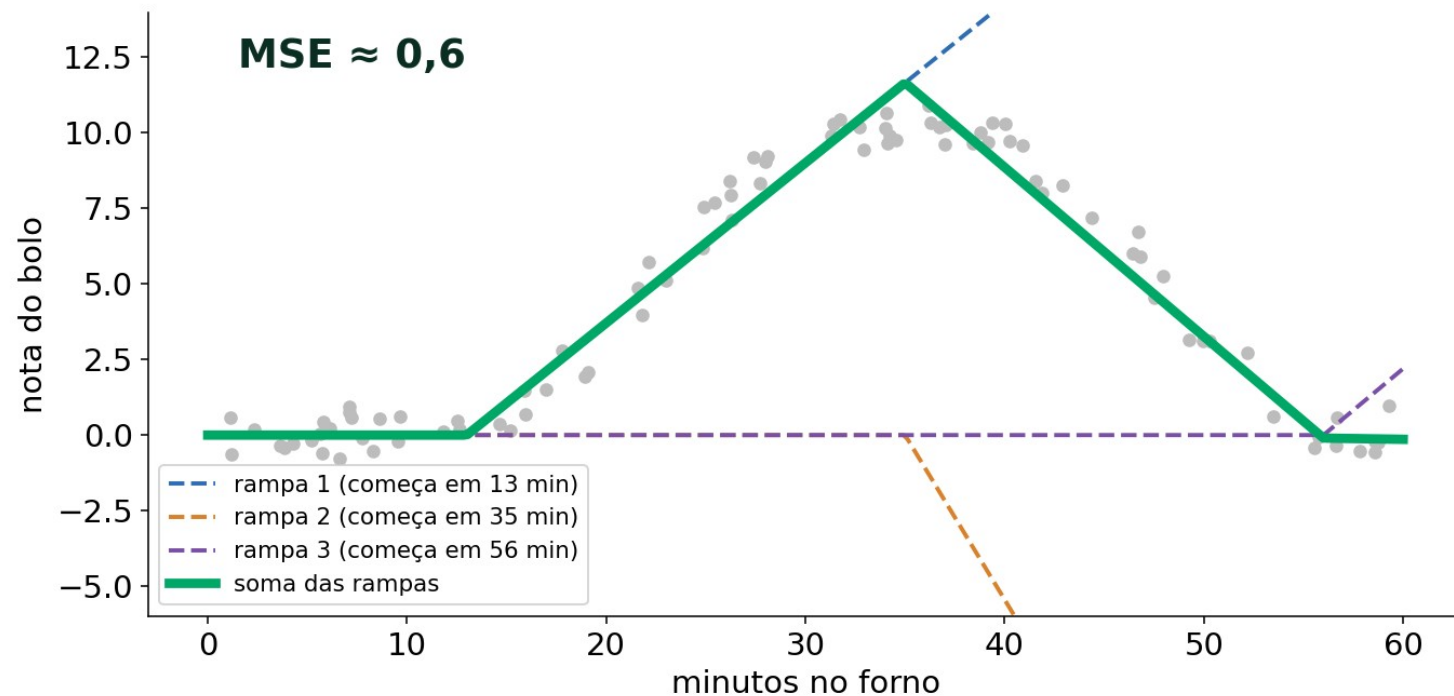
erro (MSE)

≈ 0,7

- O erro cai de $\approx 15,5$ para $\approx 0,7$
- É a menor quantidade que permite subir e voltar a descer



Três dobras: 4 trechos retos



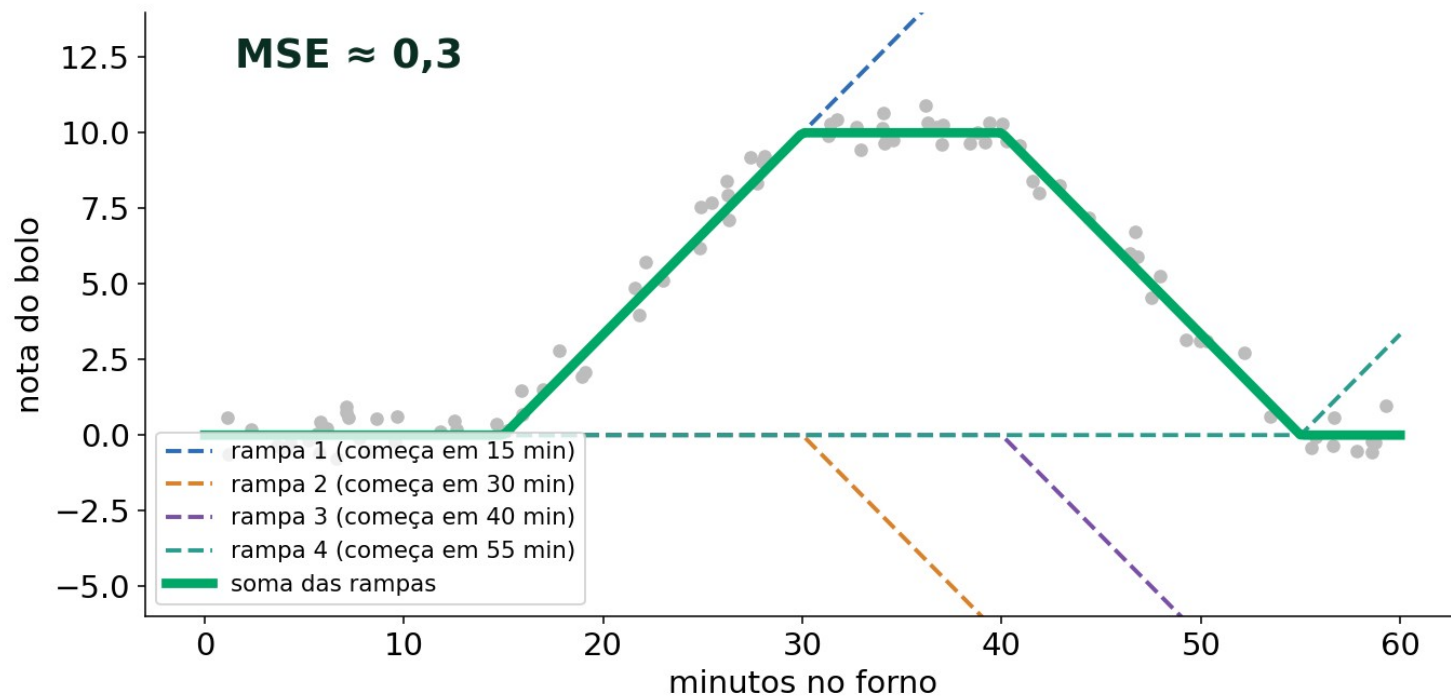
erro (MSE)

$\approx 0,6$

- Ganho pequeno em relação a duas dobras
- O que resta é principalmente o ruído dos dados



Quatro dobras: 5 trechos retos



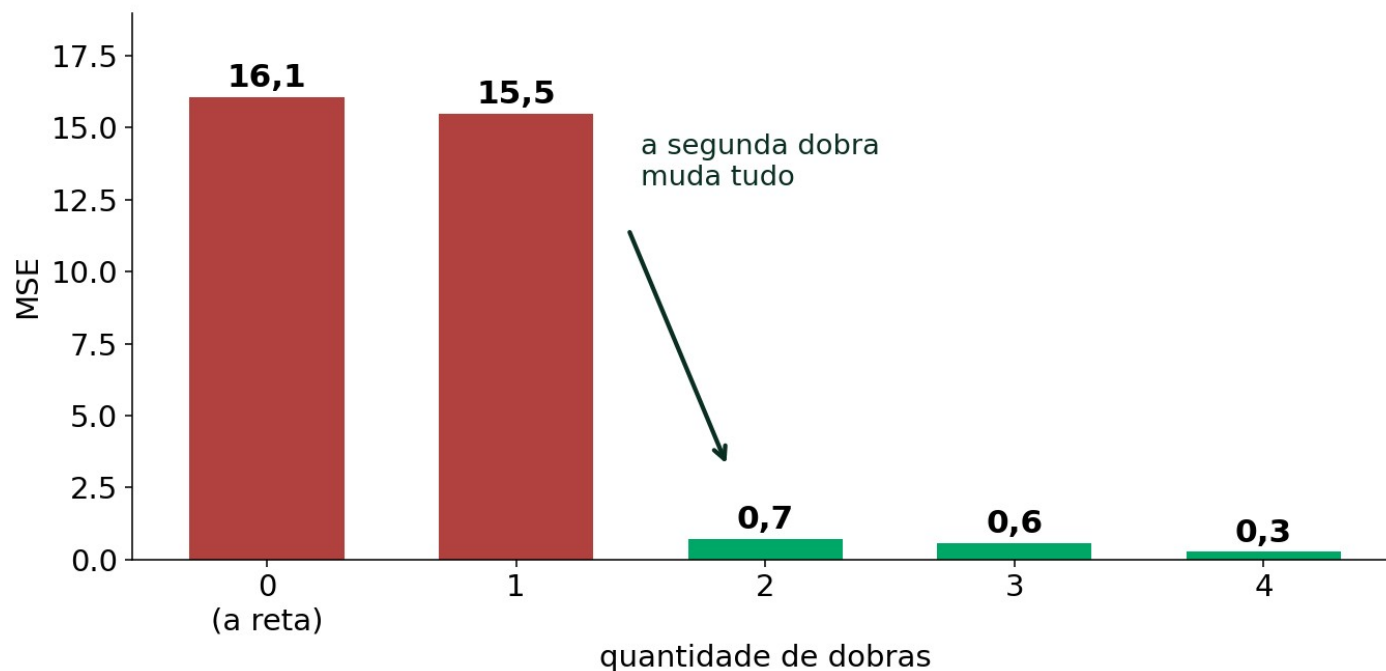
erro (MSE)

$\approx 0,3$

- Acompanha também o topo achatado
- Cada dobra nova ajuda cada vez menos



Resumo: erro por quantidade de dobras

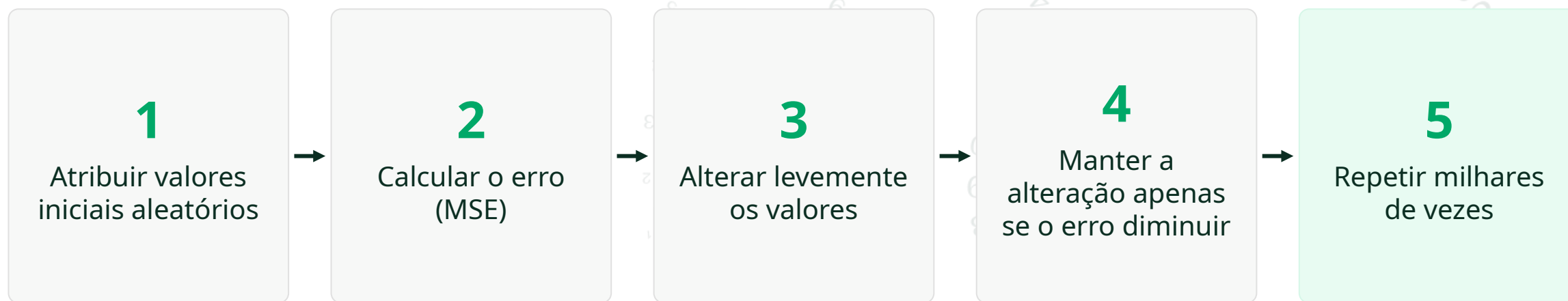


- 0 dobras (reta): 16,1
- 1 dobra: 15,5, quase não ajuda
- 2 dobras: 0,7, a queda decisiva
- 3 e 4 dobras: ganhos menores

n dobras $\rightarrow n + 1$ trechos retos

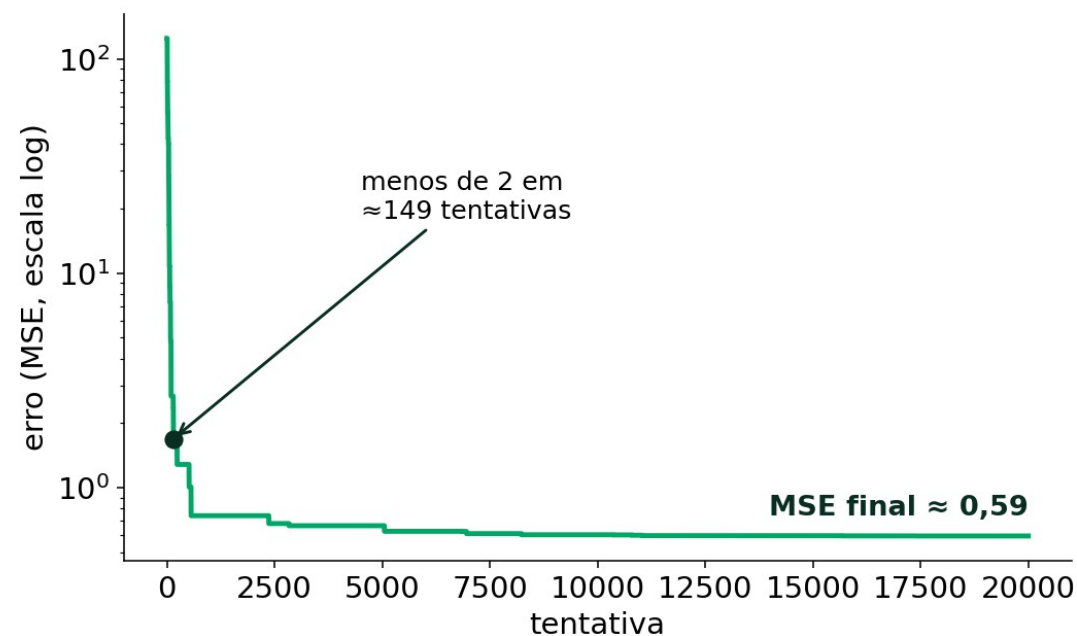
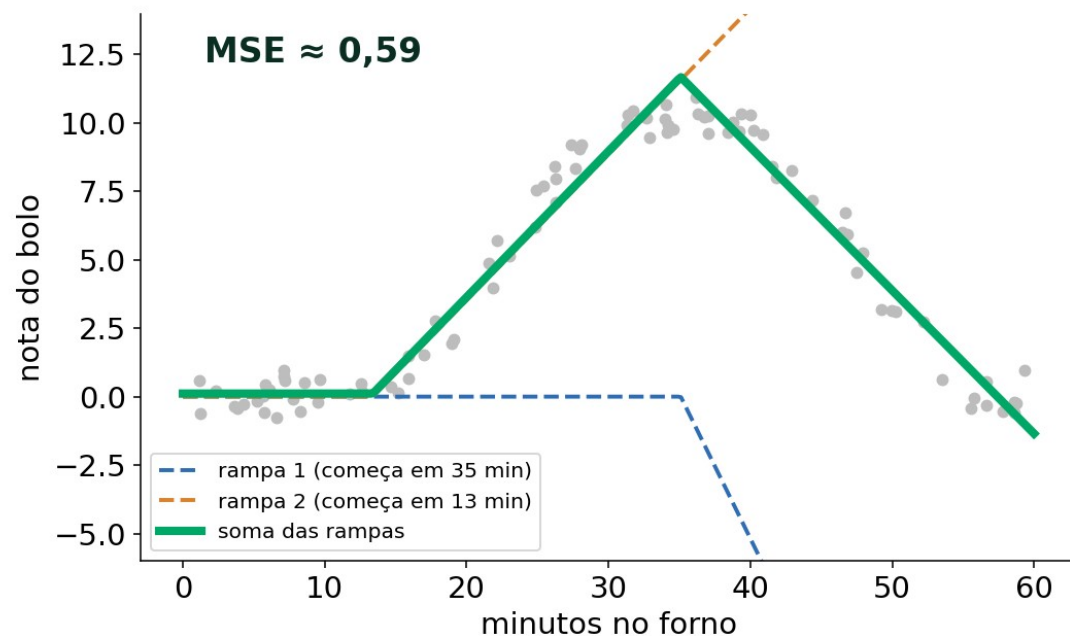
O computador procura os valores por tentativa e erro

Com duas rampas é possível estimar os valores manualmente; com dezenas, não.



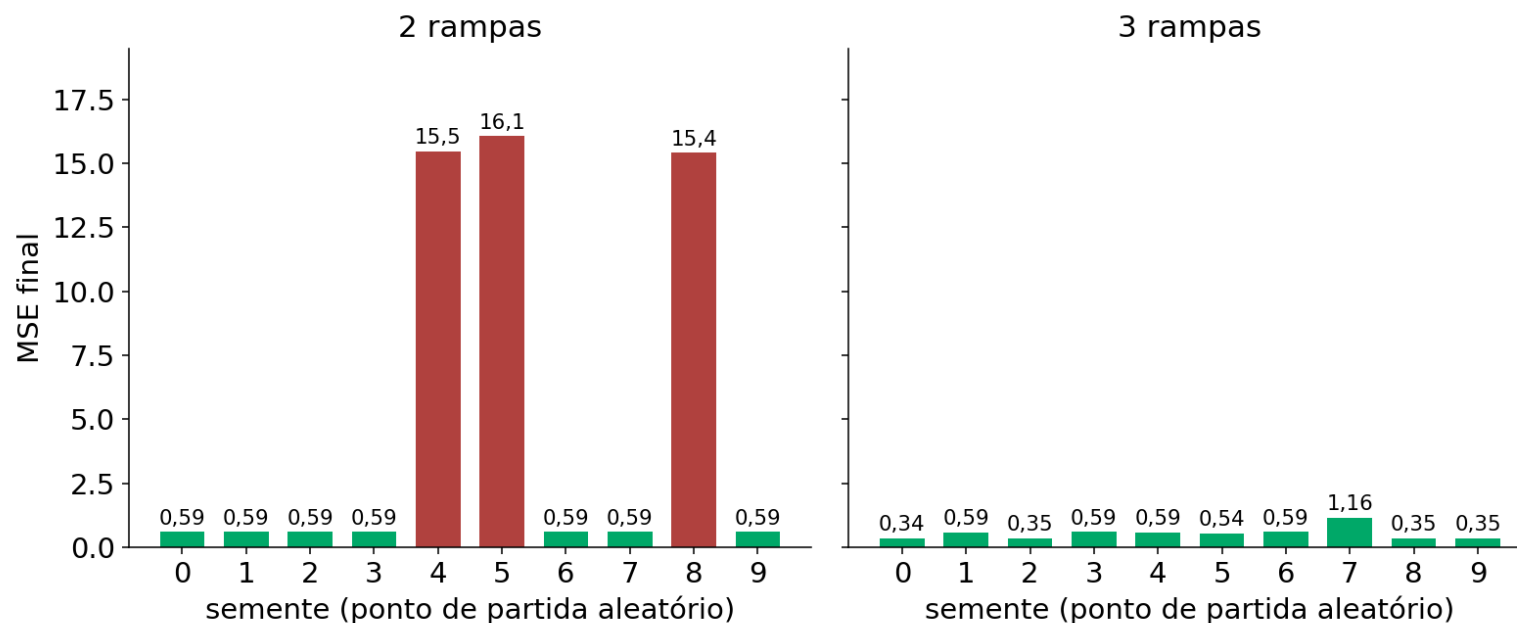
Esse procedimento é, em essência, o treinamento de um modelo.

Resultado da busca (semente 0, 2 rampas)



Rampas encontradas em ≈ 13 e 35 minutos, próximas das ajustadas manualmente (MSE $\approx 0,59$). O erro inicial supera 100 e cai para menos de 2 em ≈ 149 tentativas.

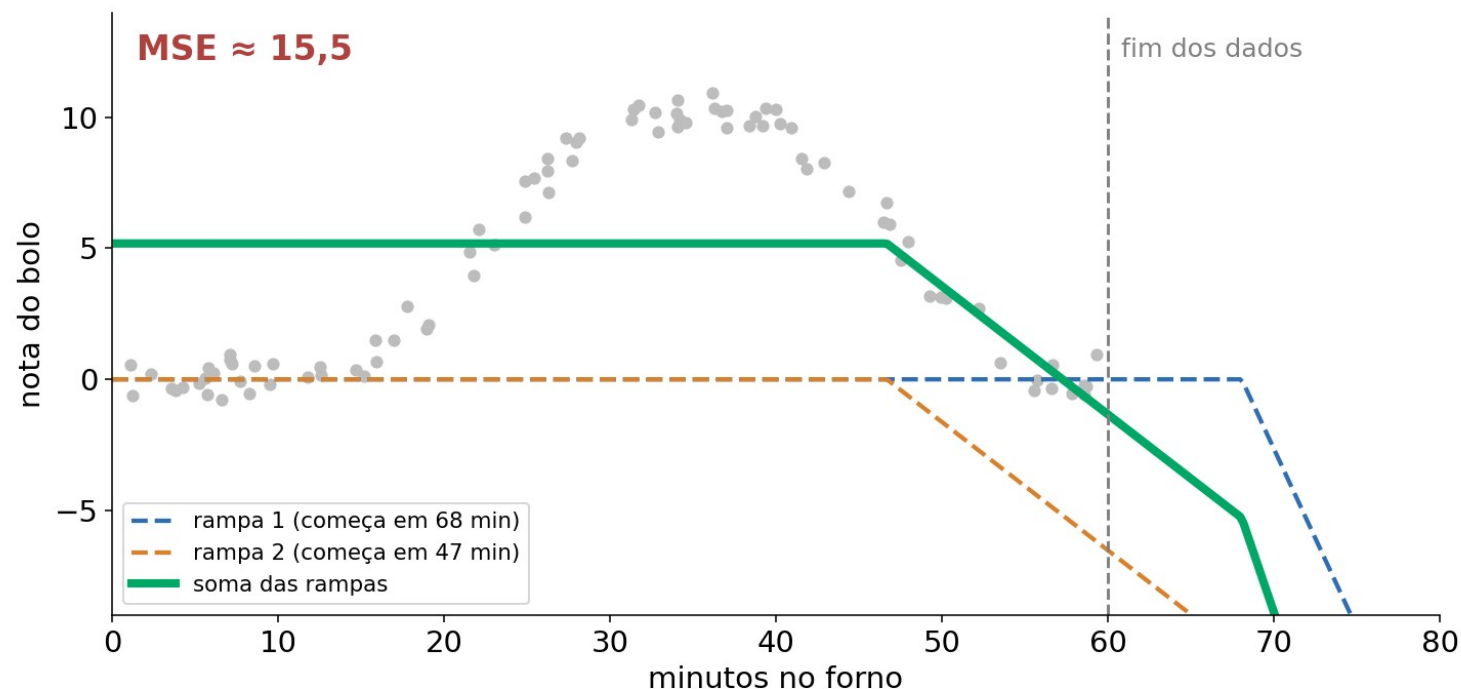
Dez execuções com pontos de partida diferentes



- 2 rampas: 7 de 10 execuções chegam a $MSE \approx 0,59$
- 3 execuções ficam entre 15 e 16, equivalente a 1 dobra ou à reta
- 3 rampas: todas abaixo de 1,2

Mais rampas aumentam a chance de sucesso.

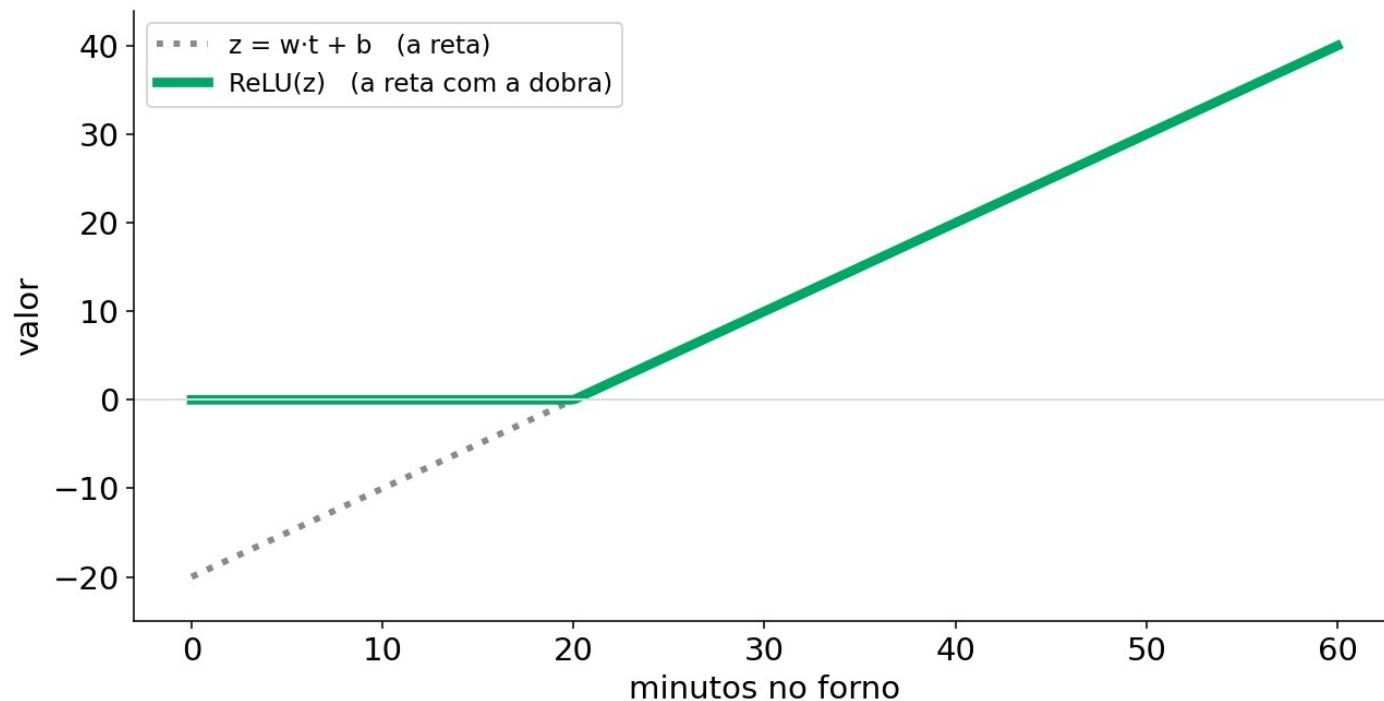
Por que algumas execuções travam



- Na semente 4, a rampa 1 começou em ≈ 68 min, fora da faixa dos dados
- Ela nunca influencia o erro: pequenas alterações não mudam nada
- A busca fica sem indicação de direção
- Em redes neurais, o fenômeno é chamado de “ReLU morto”

Resultado: ajuste equivalente a 1 dobra ($MSE \approx 15,5$).

A rampa tem outro nome: neurônio



- Uma rampa é uma reta seguida de uma dobra
- $\mathbf{z = w \cdot t + b}$ (a mesma expressão da Regressão Linear)
- **ReLU(z)**: valores negativos viram zero

Reta + dobra = neurônio artificial

No notebook: rampa e neurônio são equivalentes

```
def relu(z):  
    return np.maximum(0, z)  
  
def neuronio(t, w, b):  
    return relu(w * t + b)  
  
np.allclose(rampa(t, 20, 0.5),  
            0.5 * neuronio(t, w=1, b=-20)) # True
```

- A última linha imprime True
- Uma rampa com início em 20 min e inclinação 0,5 equivale a um neurônio com $w = 1$ e $b = -20$, multiplicado por um peso de saída 0,5

Uma rampa e um neurônio são equivalentes.

Nota sobre os parâmetros

Na rede, cada neurônio guarda w e b , e a saída guarda um peso v por neurônio e um nível base c . Os dois números da rampa são combinações desses valores:

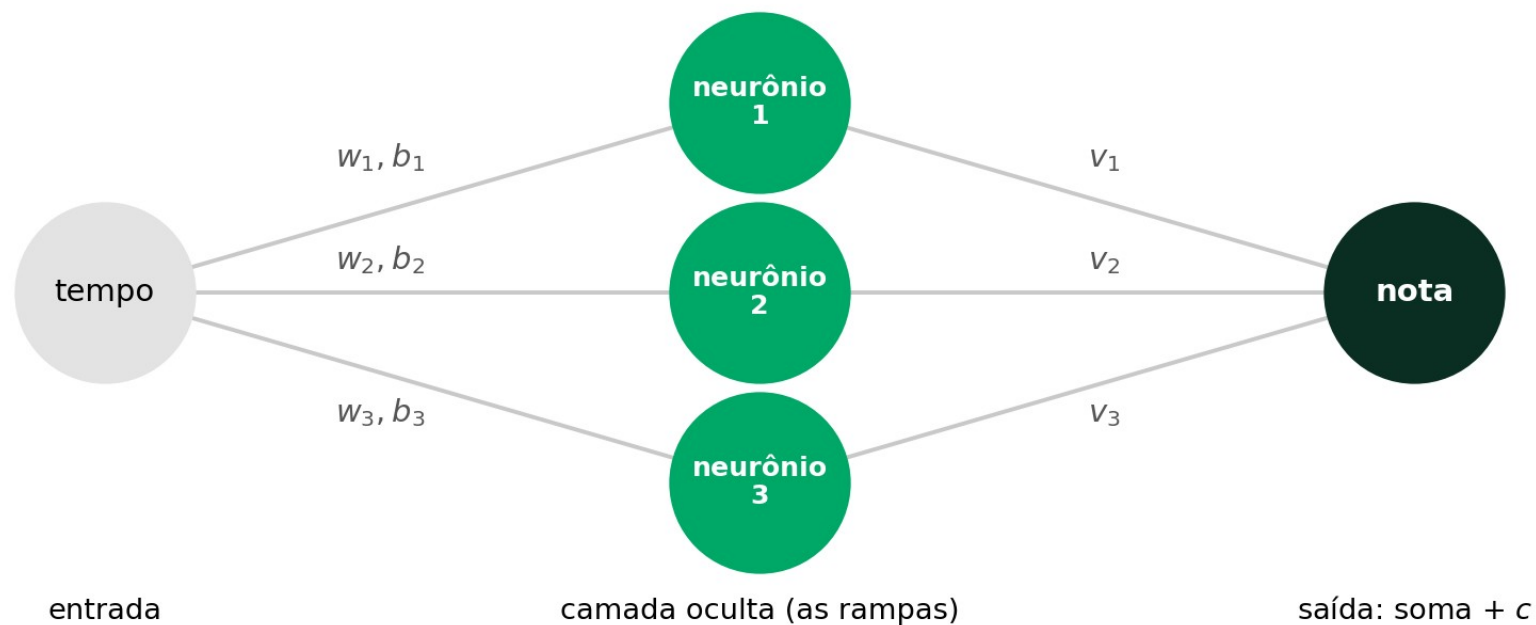
$$\text{onde} = -b / w$$

$$\text{inclinação} = v \cdot w$$

Na rede, a rampa também pode ficar virada para o outro lado: ativar-se antes de certo minuto, em vez de depois.



A rede completa



$$\text{nota} = c + v_1 \cdot \text{ReLU}(w_1 \cdot t + b_1) + v_2 \cdot \text{ReLU}(w_2 \cdot t + b_2) + v_3 \cdot \text{ReLU}(w_3 \cdot t + b_3)$$



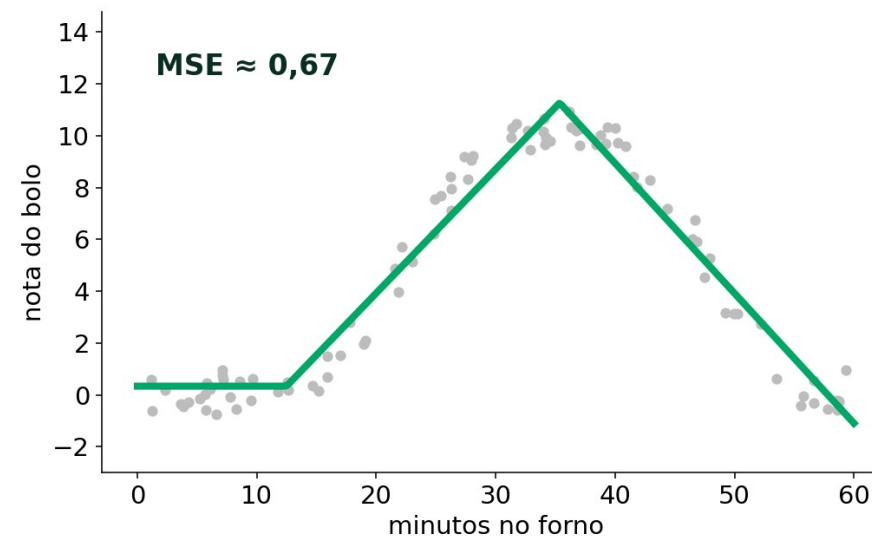
Correspondência com a terminologia

O que foi construído	Terminologia
Uma rampa (reta + dobra)	neurônio
A “dobra” (valores negativos viram zero)	função de ativação (ReLU)
Os números de cada rampa	pesos (w, b, v, c)
O grupo de rampas lado a lado	camada oculta
A soma das rampas que produz a nota	camada de saída
O computador tentando e mantendo o que reduz o erro	treinamento
A escolha de quantas rampas usar	hiperparâmetro (número de neurônios)

Implementação com o MLPRegressor

```
from sklearn.neural_network import MLPRegressor

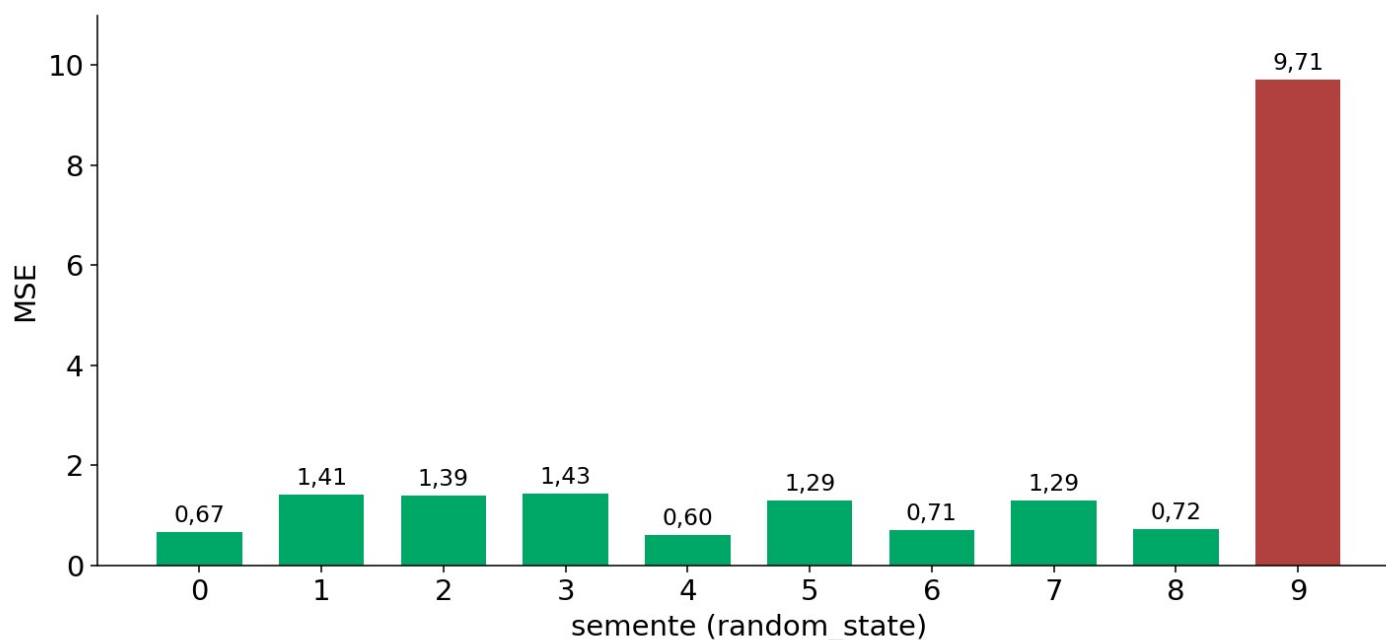
rede = MLPRegressor(hidden_layer_sizes=(3,),
                    activation="relu",
                    max_iter=20000, random_state=0)
rede.fit(X_norm, nota)
```



O usuário escolhe apenas quantos neurônios a camada oculta terá (hidden_layer_sizes).

MSE $\approx 0,67$ · 10 valores aprendidos: 3 pesos w , 3 vieses b , 3 pesos v e 1 viés c

Dez sementes no MLPRegressor



- 9 de 10 execuções: MSE entre $\approx 0,6$ e $1,4$
- 1 execução permanece em $\approx 9,7$
- Como no Passo 6, a rede começa com valores aleatórios e pode não convergir

Por isso o `random_state` é fixado ao treinar redes neurais.

O que foi construído

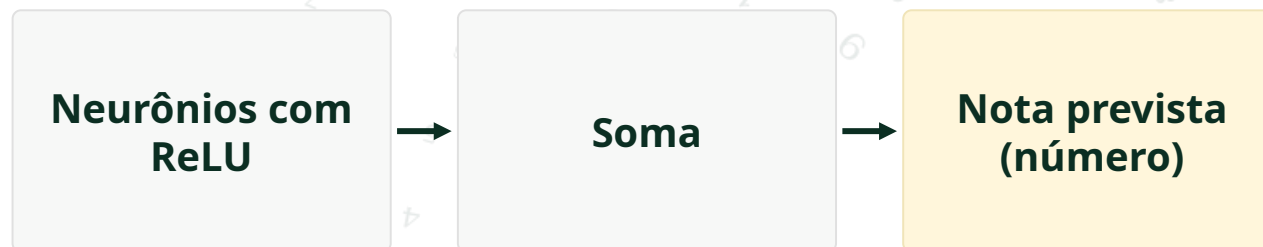
- 1 Havia um problema que a reta não resolvia: a nota precisava subir e depois voltar a descer.
- 2 A Regressão Polinomial resolve o problema dentro da faixa dos dados, mas diverge fora dela e escala mal com o número de entradas.
- 3 A alternativa é emendar rampas. Cada rampa é um neurônio; as rampas lado a lado formam a camada oculta.
- 4 Os números das rampas são os pesos. Não são escolhidos manualmente: o computador os treina, mantendo as alterações que reduzem o erro.
- 5 Para prever um número (regressão), a saída é a soma direta. Para prever uma categoria (classificação), apenas a última camada muda: a soma passa pela função sigmoide e vira probabilidade.



Regressão e classificação: só a última camada muda

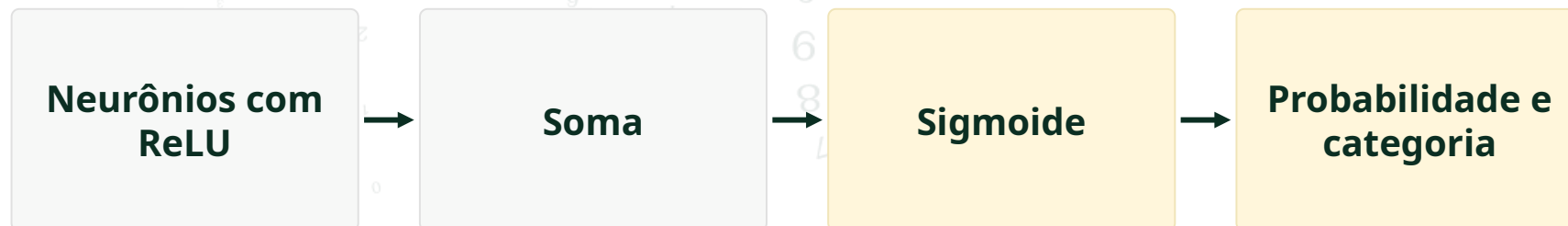
Regressão

prever um número



Classificação

prever uma categoria



O corpo da rede é o mesmo; muda apenas a saída (em amarelo).

PRÓXIMA ETAPA

A mesma rede, escrita em Keras

Neurônio, camada, pesos e treinamento: os conceitos permanecem; muda a ferramenta.

Continuação: implementação em Keras

